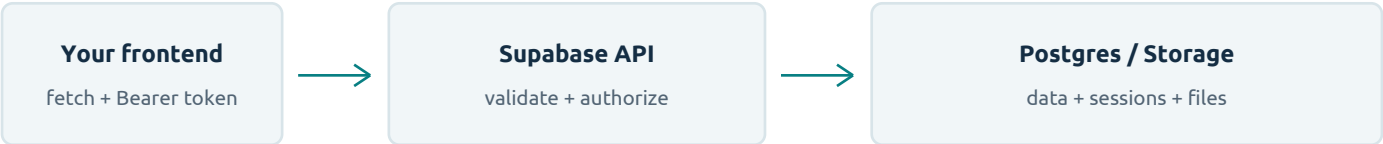


SUPABASE EDGE / TYPESCRIPT / POSTGRES + STORAGE

Frontend API Guide

Complete integration reference - 35 operations - API v1.0.0

A practical guide to integrating a local frontend with a real Supabase API. Start with a user session, learn predictable REST calls, and use the endpoint reference when building forms, lists, and uploads.



Start in five steps

1. Use the live Supabase API base URL, including /functions/v1/backend. Use http://localhost:8787 only for isolated backend development.
2. Set your public frontend base URL and allow your exact frontend origin in backend CORS configuration.
3. Register or log in. Keep the returned access and refresh tokens in memory.
4. Send Authorization: Bearer <ACCESS_TOKEN>. Parse data on success and error.code on failure.
5. Use pages, filters and PATCH to build UI flows; upload File/Blob bytes directly for private images.

Connection addresses

Environment	Base URL and role
Live API	https://fvhpmujmbavwdpjoyyal.supabase.co/functions/v1/backend
Local backend	http://localhost:8787 - isolated PostgreSQL development data
Reference and contract	Local /docs or portable docs/index.html; live /openapi.json - OpenAPI3.1. Endpoint paths append to the full base URL.

The base URL has no /api/v1 suffix, query, fragment or credentials. Examples use placeholders, never built-in credentials. Live accounts and local seed accounts are separate.

Contents

Frontend setup and first calls	2
Responses, lists and errors	2
Authentication and private files	4
Complete endpoint reference	5
Response schema catalog	22
Local development and deployment	25
Configuration and maintenance reference	26

</> Frontend setup and first calls

Your frontend can remain on your machine while the API is hosted on Supabase. In the frontend project, create `.env.local` with the variable for your framework, then restart the dev server. Published builds must be rebuilt after changing this value.

```
# Vite - public API URL only
VITE_API_BASE_URL=https://fvhpmujmbavwdpjyayal.supabase.co/functions/v1/backend
# Next.js - use this name instead
NEXT_PUBLIC_API_BASE_URL=https://fvhpmujmbavwdpjyayal.supabase.co/functions/v1/backend
```

Never expose API keys, Supabase account tokens, `JWT_SECRET`, `PASSWORD_PEPPER` or `IP_HASH_SECRET` through `VITE_ / NEXT_PUBLIC_` variables. Framework public variables are visible in the browser bundle.

Configure exact CORS origins

Allow only the frontend origins you use. `localhost` and `127.0.0.1` are different origins; ports and schemes also matter. Production allows HTTP only for configured loopback hosts. The hosted API itself requires HTTPS; published frontend origins require HTTPS. Serve HTML with a dev server, rather than `file://`.

```
ALLOWED_ORIGINS=http://localhost:3000,http://127.0.0.1:3000,
http://localhost:5173,http://127.0.0.1:5173,
http://localhost:5500,http://127.0.0.1:5500
# Put the selected origins on ONE comma-separated configuration line.
```

Sign in and read projects with fetch

```
const baseUrl = import.meta.env.VITE_API_BASE_URL;
// Next.js client: process.env.NEXT_PUBLIC_API_BASE_URL
const loginResponse = await fetch(`${baseUrl}/api/v1/auth/login`, {
  method: 'POST', credentials: 'omit',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password }),
});
const login = await loginResponse.json();
if (!loginResponse.ok) throw new Error(login.error.message);
let { accessToken, refreshToken } = login.data; // Memory only.
const response = await fetch(`${baseUrl}/api/v1/projects?page=1&limit=20`, {
  credentials: 'omit', headers: { Authorization: `Bearer ${accessToken}` },
});
const result = await response.json();
if (!response.ok) throw new Error(result.error.message);
renderProjects(result.data, result.meta); // Your UI function.
```

`fetch` rejects for network/transport failures, not HTTP 4xx/5xx. Check `response.ok`. Retain `requestId` for support; do not log tokens or raw credentials. Copy `examples/frontend.ts` into your frontend for typed CRUD, timeouts, `AbortSignal` support and serialized refresh.

Create, update and remove a resource

```
import { LearningApi, ApiClientError } from './frontend';
const api = new LearningApi(import.meta.env.VITE_API_BASE_URL);
await api.login(email, password);
const project = await api.createProject({ name: 'My first project' });
const task = await api.createTask({ title: 'Build the list', projectId: project.id });
await api.updateTask(task.id, { status: 'done' });
const page = await api.tasks({ projectId: project.id, page: 1, limit: 10 });
renderTasks(page.items);
await api.deleteTask(task.id);
```

Represent loading, empty, success and error states separately. Disable repeated submissions while writes are pending. Debounce search and abort obsolete reads. Render text with framework interpolation or `textContent`, never HTML injection helpers.

📄 Responses, lists and errors

JSON resource responses use the same envelope. Lists return a data array plus meta; single resources return an object. Session lists return `data.sessions` and key lists return `data.items`. Creates return 201; normal reads, PATCH and DELETE return 200. Resource deletes return `{id, deleted:true}`; revocations return `{revoked:true}`.

```
{
  "success": true,
  "data": [],
  "meta": {
    "page": 1, "limit": 20, "total": 0, "totalPages": 0, "hasNext": false,
    "hasPrevious": false
  },
  "requestId": "<REQUEST_ID>"
}
```

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR", "message": "Some fields are invalid",
    "details": [
      {
        "field": "name",
        "message": "Use plain text without markup or control characters"
      }
    ]
  },
  "requestId": "<REQUEST_ID>"
}
```

The X-Request-Id header matches requestId. OpenAPI JSON, reference HTML/CSS, successful binary downloads and successful 204 CORS preflight do not use this envelope. Download failures still return the JSON error envelope.

HTTP	Meaning	Frontend action
400	Malformed JSON/body/header, ambiguous credentials or unexpected DELETE body	Fix the request.
401	Missing, invalid, expired or revoked credentials	Refresh once; sign in if refresh fails.
403	Origin, role, scope or registration policy denies access	Explain permission failure; do not retry blindly.
404	Missing endpoint or resource, including another owner's resource	Show a missing resource state.
409	Duplicate email, relation conflict or active API-key cap	Resolve the conflict before resubmitting.
413 / 415	Body too large / unsupported media type or content encoding	Reduce bytes / use the documented Content-Type.
422	Invalid fields, UUID, calendar date or query parameters	Show field messages from error.details.
429	Rate limit exceeded; response includes Retry-After	Respect the delay and avoid synchronized retries.
500 / 503	Internal failure / temporary service or configuration failure	Keep requestId; check write state before retrying.

Pagination and search

Default page=1, limit=20, sort=createdAt, order=desc. Page is 1-10000 and limit is 1-100. q is optional trimmed plain text with 1-100 characters; omit empty search. Unknown or duplicate list parameters return 422. Search matches literal substrings: % and _ are escaped, not wildcard operators. A page past the end returns an empty successful list.

Collection	Filters	Sort fields / search
Projects	status: active archived	createdAt, updatedAt, name / name + description
Tasks	status: todo in_progress done; priority: low medium high; projectId: UUID	createdAt, updatedAt, title, dueDate / title + description
Files	contentType: image/png image/jpeg image/webp	createdAt, filename, size / filename
Users (admin)	role: member admin	createdAt, name, email / name + email
Audit (admin)	action: text; actorId: UUID	createdAt / no q search

```
const query = new URLSearchParams({ page: '1', limit: '10', order: 'asc' });
if (search.trim()) query.set('q', search.trim());
const response = await fetch(`${baseUrl}/api/v1/tasks?${query}`, {
  headers: { Authorization: `Bearer ${accessToken}` }, credentials: 'omit',
});
```

PATCH changes only supplied fields and must include at least one supported field. Omit a field to keep it; pass null to clear task projectId or dueDate. Dates use YYYY-MM-DD. Deleting a project keeps tasks and sets their projectId to null. Offset pages can move during concurrent changes; use a cursor-based module for future large feeds.



Authentication and private files

Caller	Authentication header	Permissions
Browser user	Authorization: Bearer <ACCESS_TOKEN>	Owned resources; session/profile management; admin lists require admin role.
Trusted server / BFF	X-API-Key: <API_KEY>	Owned projects/tasks/files only, restricted by explicit scopes.
Public endpoint	No authentication header	Registration, login, refresh and system routes only.

Use one authentication method per request; sending both returns 400. Never embed an API key in browser or mobile code. CORS does not keep client-side secrets private. A member and an administrator both see only their own projects, tasks and files. Another owner's ID returns 404.

Scope	What it authorizes
resources:read	GET project/task collection and detail routes
resources:write	POST/PATCH/DELETE project/task routes
files:read	GET file metadata, collection and private content
files:write	POST upload and DELETE file routes

Write does not imply read. Create/manage keys with a Bearer session; plaintext keys are returned once. Keys expire in 1-90 days and are capped at 20 active keys per user. Normal browser integrations should use their user's session, never a shared key.

Access, refresh and logout lifecycle

Access JWTs expire after expiresIn seconds (default 900). Sessions expire absolutely after seven days by default; refreshing does not extend sessionExpiresAt. Up to 20 active sessions are retained; another login evicts the oldest. PostgreSQL checks current user, role, session and key state on protected requests, so logout, disabling, role changes and revocation take effect before JWT expiry.

```
// Serialize this exchange: reuse of the old token revokes the session.
const response = await fetch(`${baseUrl}/api/v1/auth/refresh`, {
  method: 'POST', credentials: 'omit',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ refreshToken }),
});
const result = await response.json();
if (!response.ok) { accessToken = ''; refreshToken = ''; throw new Error('Sign in again'); }
({ accessToken, refreshToken } = result.data); // Store BOTH replacements immediately.
// On logout, POST /api/v1/auth/logout with Bearer auth, then clear both tokens.
```

Keep tokens in memory, not URLs, logs or persistent browser storage. The sample client serializes refresh within one instance; coordinate multiple tabs/contexts or use a BFF. A lost rotation response can require sign-in again. Do not blindly retry write timeouts, 429 or 5xx: the first mutation may already have committed. No authentication cookies are set; credentials: omit is appropriate.

Upload and download an image

Send a File or Blob as the raw body, not multipart FormData or base64. The current upload cap is 5 MiB. Static PNG, JPEG and WebP are accepted; signatures, structure, extensions and dimensions are checked. Each dimension is at most 16384 pixels and total pixels at most 40 million. APNG, animated WebP, SVG and HTML are rejected.

```
const file = input.files?.[0];
if (file) {
  const response = await fetch(`${baseUrl}/api/v1/files`, {
    method: 'POST', credentials: 'omit',
    headers: { Authorization: `Bearer ${accessToken}`,
      'Content-Type': file.type, 'X-File-Name': file.name },
    body: file,
  });
  const result = await response.json();
  if (!response.ok) throw new Error(result.error.message);
  const content = await fetch(`${baseUrl}/${result.data.downloadUrl}`, {
    headers: { Authorization: `Bearer ${accessToken}` }, credentials: 'omit',
  });
  if (!content.ok) throw new Error((await content.json()).error.message);
  const objectUrl = URL.createObjectURL(await content.blob());
  image.src = objectUrl;
}
```

```
image.onload = () => URL.revokeObjectURL(objectUrl);
}
```

Filenames use bounded ASCII characters without paths; spaces become underscores and the extension must match MIME. Storage remains private. An img src cannot attach Bearer auth; use an authorized fetch and a Blob URL. Downloads are attachments with nosniff and private/no-store caching. Validation is structural, not malware scanning or full image decoding.

☰ Complete endpoint reference

Every HTTP operation in docs/openapi.json appears below. Set API_BASE_URL to your selected base URL. ACCESS_TOKEN, REFRESH_TOKEN, API_KEY, RESOURCE_ID and filenames are placeholders or your own values. Request examples choose Bearer auth when supported; trusted server callers may replace that header with X-API-Key when the listed scope allows it. Error status numbers refer to the common error table.

AUTHENTICATION

POST /api/v1/auth/register

Register a member and create a session

AUTH Public; no auth header

Public registration can be disabled by configuration. Unknown fields, including role, are rejected. New accounts always have the member role. Registration name is limited to 80 characters.

JSON body (* required)	Type	Rules / defaults
email *	string (email)	Length 0 to 254; Trimmed and normalized to lowercase.
password *	string	Length 15 to 128; Write only; Passwords are never trimmed or returned. Maximum 256 UTF-8 bytes, no control characters.
name *	string	Length 1 to 80; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.

JSON object rejects unknown fields.

Request example

```
curl -X POST "${API_BASE_URL}/api/v1/auth/register" \
  -H 'Content-Type: application/json' \
  --data-binary @- <<'JSON'
{"email": "learner@example.com", "name": "API Learner", "password": "<YOUR_LONG_PASSPHRASE>"}
JSON
```

Response example - HTTP 201

```
{
  "success": true,
  "data": {
    "user": {
      "id": "2cd65e90-ce9f-42e4-a7f4-1e6dc8dba84b", "email": "learner@example.com",
      "name": "API Learner", "role": "member",
      "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
    },
    "accessToken": "<ACCESS_TOKEN>", "refreshToken": "<REFRESH_TOKEN>", "tokenType": "Bearer",
    "expiresIn": 900, "sessionExpiresAt": "2026-01-08T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: AuthTokens; see the schema catalog. All examples are illustrative, not credentials.

ERROR STATUSES 400, 403, 409, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

POST /api/v1/auth/login

Create a session using email and password

AUTH Public; no auth header

A user may have at most 20 active sessions; creating a new session beyond the cap atomically revokes the oldest session.

JSON body (* required)	Type	Rules / defaults
email *	string (email)	Length 0 to 254; Trimmed and normalized to lowercase.
password *	string	Length 15 to 128; Write only; Passwords are never trimmed or returned. Maximum 256 UTF-8 bytes, no control characters.

JSON object rejects unknown fields.

Request example

```
curl -X POST "${API_BASE_URL}/api/v1/auth/login" \
  -H 'Content-Type: application/json' \
  --data-binary @- <<'JSON'
{"email": "learner@example.com", "password": "<YOUR_LONG_PASSPHRASE>"}
JSON
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "user": {
      "id": "2cd65e90-ce9f-42e4-a7f4-1e6dc8dba84b", "email": "learner@example.com",
      "name": "API Learner", "role": "member",
      "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
    },
    "accessToken": "<ACCESS_TOKEN>", "refreshToken": "<REFRESH_TOKEN>", "tokenType": "Bearer",
    "expiresIn": 900, "sessionExpiresAt": "2026-01-08T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: AuthTokens; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

POST

/api/v1/auth/refresh

Rotate a refresh token and obtain a new access token

AUTH Public; no auth header

Send the refresh token in JSON. It can be used once. Store the returned replacement. Coordinate refreshes in the frontend so parallel requests do not race. An invalid, expired, or reused token returns 401. Replay of a consumed token revokes the entire session.

JSON body (* required)	Type	Rules / defaults
refreshToken *	string	Write only; Validated token format

JSON object rejects unknown fields.

Request example

```
curl -X POST "${API_BASE_URL}/api/v1/auth/refresh" \
  -H 'Content-Type: application/json' \
  --data-binary @- <<'JSON'
{"refreshToken": "<REFRESH_TOKEN>"}
JSON
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "user": {
      "id": "2cd65e90-ce9f-42e4-a7f4-1e6dc8dba84b", "email": "learner@example.com",
      "name": "API Learner", "role": "member",
      "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
    },
    "accessToken": "<ACCESS_TOKEN>", "refreshToken": "<REFRESH_TOKEN>", "tokenType": "Bearer",
    "expiresIn": 900, "sessionExpiresAt": "2026-01-08T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: AuthTokens; see the schema catalog. All examples are illustrative, not credentials.

ERROR STATUSES 400, 401, 403, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

POST /api/v1/auth/logout

Revoke the current session

AUTH Bearer access token only

Send the current access token. No request body is needed. Its access and refresh tokens stop authorizing requests after revocation.

REQUEST BODY None. Do not send a body.

Request example

```
curl -X POST "${API_BASE_URL}/api/v1/auth/logout" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{"success": true, "data": {"revoked": true}, "requestId": "req_example_01"}
```

RESPONSE TYPE data: Revoked; see the schema catalog. All examples are illustrative, not credentials.

ERROR STATUSES 400, 401, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET /api/v1/auth/me

Get the current authenticated user

AUTH Bearer access token only

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/auth/me" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "user": {
      "id": "2cd65e90-ce9f-42e4-a7f4-1e6dc8dba84b", "email": "learner@example.com",
      "name": "API Learner", "role": "member",
      "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
    },
    "requestId": "req_example_01"
  }
}
```

RESPONSE TYPE data: object; see the schema catalog. All examples are illustrative, not credentials.

ERROR STATUSES 400, 401, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET /api/v1/auth/sessions

List your active sessions

AUTH Bearer access token only

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/auth/sessions" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "sessions": [
      {
        "id": "b70efcda-37b8-47cc-915d-88e34902ae75",
        "createdAt": "2026-01-01T10:00:00.000Z",
        "expiresAt": "2026-01-08T10:00:00.000Z",
        "lastSeenAt": "2026-01-01T10:00:00.000Z",
        "current": true
      }
    ]
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: object; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

DELETE /api/v1/auth/sessions/{id}

Revoke one of your sessions

AUTH Bearer access token only

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

REQUEST BODY None. Do not send a body.

Request example

```
curl -X DELETE "${API_BASE_URL}/api/v1/auth/sessions/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{"success": true, "data": {"revoked": true}, "requestId": "req_example_01"}
```

RESPONSE TYPE data: Revoked; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

PROJECTS

GET /api/v1/projects

List, filter, search, and sort your projects

AUTH Bearer access token OR trusted-server API key; API key scope resources:read
Requires resources:read for API keys. Search matches project name and description as literal text. Other users' projects never appear.

Parameter (* required)	Location / type	Allowed values and behavior
page	query / integer	Range 1 to 10000; Default: 1
limit	query / integer	Range 1 to 100; Default: 20
q	query / string	Length 1 to 100; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.; Literal plain-text search. Omit for no search. SQL wildcard characters have no special meaning.
order	query / string	Allowed: asc, desc; Default: "desc"
sort	query / string	Allowed: createdAt, updatedAt, name; Default: "createdAt"; Allowlisted field; arbitrary SQL column names are rejected.
status	query / string	Allowed: active, archived

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/projects?page=1&limit=20&order=desc&sort=createdAt&q=learn" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": [
    {
      "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "name": "Learning API integration",
      "description": "Practice fetching a real API", "status": "active",
      "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
    }
  ],
  "requestId": "req_example_01",
  "meta": {
    "page": 1, "limit": 20, "total": 1, "totalPages": 1, "hasNext": false,
    "hasPrevious": false
  }
}
```

RESPONSE TYPE data: Project[]; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

POST /api/v1/projects

Create a project

AUTH Bearer access token OR trusted-server API key; API key scope resources:write
Requires resources:write for API keys.

JSON body (* required)	Type	Rules / defaults
name *	string	Length 1 to 120; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
description	string	Length 0 to 2000; Default: ""; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
status	string	Allowed: active, archived; Default: "active"

JSON object rejects unknown fields.

Request example

```
curl -X POST "${API_BASE_URL}/api/v1/projects" \
-H "Authorization: Bearer ${ACCESS_TOKEN}" \
-H 'Content-Type: application/json' \
--data-binary @- <<'JSON'
{
  "name": "Learning API integration", "description": "Practice fetching a real API",
  "status": "active"
}
JSON
```

Response example - HTTP 201

```
{
  "success": true,
  "data": {
    "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "name": "Learning API integration",
    "description": "Practice fetching a real API", "status": "active",
    "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Project; see the schema catalog. All examples are illustrative, not credentials.
RESPONSE HEADERS Location
ERROR STATUSES 400, 401, 403, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET /api/v1/projects/{id}

Read a project you own

AUTH Bearer access token OR trusted-server API key; API key scope resources:read

Requires resources:read for API keys.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/projects/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "name": "Learning API integration",
    "description": "Practice fetching a real API", "status": "active",
    "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Project; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

PATCH

/api/v1/projects/{id}

Update supplied project fields

AUTH Bearer access token OR trusted-server API key; API key scope resources:write
Requires resources:write for API keys. Omitted fields remain unchanged.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

JSON body (* required)	Type	Rules / defaults
name	string	Length 1 to 120; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
description	string	Length 0 to 2000; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
status	string	Allowed: active, archived

JSON object rejects unknown fields. Supply at least one field; omitted values remain unchanged.

Request example

```
curl -X PATCH "${API_BASE_URL}/api/v1/projects/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}" \
-H 'Content-Type: application/json' \
--data-binary @- <<'JSON'
{"status": "archived"}
JSON
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "name": "Learning API integration",
    "description": "Practice fetching a real API", "status": "archived",
    "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Project; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

DELETE /api/v1/projects/{id}

Delete a project you own

AUTH Bearer access token OR trusted-server API key; API key scope resources:write

Associated tasks remain; their projectId becomes null. Requires resources:write for API keys.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

REQUEST BODY None. Do not send a body.

Request example

```
curl -X DELETE "${API_BASE_URL}/api/v1/projects/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {"id": "b70efcda-37b8-47cc-915d-88e34902ae75", "deleted": true},
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Deleted; see the schema catalog. All examples are illustrative, not credentials.

ERROR STATUSES 400, 401, 403, 404, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

TASKS

GET /api/v1/tasks

List, filter, search, and sort your tasks

AUTH Bearer access token OR trusted-server API key; API key scope resources:read

Requires resources:read for API keys. Search matches task title and description as literal text.

Parameter (* required)	Location / type	Allowed values and behavior
page	query / integer	Range 1 to 10000; Default: 1
limit	query / integer	Range 1 to 100; Default: 20
q	query / string	Length 1 to 100; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.; Literal plain-text search. Omit for no search. SQL wildcard characters have no special meaning.
order	query / string	Allowed: asc, desc; Default: "desc"
sort	query / string	Allowed: createdAt, updatedAt, title, dueDate; Default: "createdAt"; Allowlisted field; arbitrary SQL column names are rejected.
status	query / string	Allowed: todo, in_progress, done
priority	query / string	Allowed: low, medium, high
projectId	query / string (uuid)	Optional; see request example.

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/tasks?page=1&limit=20&order=desc&sort=createdAt&q=learn" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": [
    {
      "id": "2fdd0cd8-74e6-4776-b509-f312dd442ae2", "title": "Build a project list",
      "description": "Fetch the first page", "status": "todo", "priority": "medium",
      "projectId": "b70efcda-37b8-47cc-915d-88e34902ae75", "dueDate": "2026-10-31",
      "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
    }
  ],
  "requestId": "req_example_01",
  "meta": {
    "page": 1, "limit": 20, "total": 1, "totalPages": 1, "hasNext": false,
    "hasPrevious": false
  }
}
```

RESPONSE TYPE data: Task[]; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

POST /api/v1/tasks

Create a task, optionally within your project

AUTH Bearer access token OR trusted-server API key; API key scope resources:write
Requires resources:write for API keys. projectId must identify a project you own.

JSON body (* required)	Type	Rules / defaults
title *	string	Length 1 to 200; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
description	string	Length 0 to 4000; Default: ""; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
status	string	Allowed: todo, in_progress, done; Default: "todo"
priority	string	Allowed: low, medium, high; Default: "medium"
projectId	string (uuid) null	Default: null; ID of a project you own, or null to remove the association.
dueDate	string (date) null	Default: null; Valid calendar date, YYYY-MM-DD, or null.

JSON object rejects unknown fields.

Request example

```
curl -X POST "${API_BASE_URL}/api/v1/tasks" \
-H "Authorization: Bearer ${ACCESS_TOKEN}" \
-H 'Content-Type: application/json' \
--data-binary @- <<'JSON'
{
  "title": "Build a project list", "description": "Fetch the first page", "status": "todo",
  "priority": "medium", "projectId": "b70efcda-37b8-47cc-915d-88e34902ae75", "dueDate": "2026-10-31"
}
JSON
```

Response example - HTTP 201

```
{
  "success": true,
  "data": {
    "id": "2fdd0cd8-74e6-4776-b509-f312dd442ae2", "title": "Build a project list",
    "description": "Fetch the first page", "status": "todo", "priority": "medium",
    "projectId": "b70efcda-37b8-47cc-915d-88e34902ae75", "dueDate": "2026-10-31",
    "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Task; see the schema catalog. All examples are illustrative, not credentials.
RESPONSE HEADERS Location
ERROR STATUSES 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET

/api/v1/tasks/{id}

Read a task you own

AUTH Bearer access token OR trusted-server API key; API key scope resources:read
Requires resources:read for API keys.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/tasks/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "id": "2fdd0cd8-74e6-4776-b509-f312dd442ae2", "title": "Build a project list",
    "description": "Fetch the first page", "status": "todo", "priority": "medium",
    "projectId": "b70efcda-37b8-47cc-915d-88e34902ae75", "dueDate": "2026-10-31",
    "createdAt": "2026-01-01T10:00:00.000Z", "updatedAt": "2026-01-01T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Task; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

PATCH

/api/v1/tasks/{id}

Update supplied task fields

AUTH Bearer access token OR trusted-server API key; API key scope resources:write
Requires resources:write for API keys. Omitted fields remain unchanged.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

JSON body (* required)	Type	Rules / defaults
title	string	Length 1 to 200; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
description	string	Length 0 to 4000; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
status	string	Allowed: todo, in_progress, done
priority	string	Allowed: low, medium, high
projectId	string (uuid) null	ID of a project you own, or null to remove the association.
dueDate	string (date) null	Valid calendar date, YYYY-MM-DD, or null.

JSON object rejects unknown fields. Supply at least one field; omitted values remain unchanged.

Request example

```
curl -X PATCH "${API_BASE_URL}/api/v1/tasks/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}" \
-H 'Content-Type: application/json' \
--data-binary @- <<'JSON'
{"status": "done", "projectId": null}
JSON
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "id": "2fdd0cd8-74e6-4776-b509-f312dd442ae2", "title": "Build a project list",
    "description": "Fetch the first page", "status": "done", "priority": "medium",
    "projectId": null, "dueDate": "2026-10-31", "createdAt": "2026-01-01T10:00:00.000Z",
    "updatedAt": "2026-01-01T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Task; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 409, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

DELETE /api/v1/tasks/{id}

Delete a task you own

AUTH Bearer access token OR trusted-server API key; API key scope resources:write
Requires resources:write for API keys.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

REQUEST BODY None. Do not send a body.

Request example

```
curl -X DELETE "${API_BASE_URL}/api/v1/tasks/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {"id": "b70efcda-37b8-47cc-915d-88e34902ae75", "deleted": true},
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Deleted; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

FILES

GET /api/v1/files

List metadata for your private files

AUTH Bearer access token OR trusted-server API key; API key scope files:read
Requires files:read for API keys. Search matches filename as literal text.

Parameter (* required)	Location / type	Allowed values and behavior
page	query / integer	Range 1 to 10000; Default: 1
limit	query / integer	Range 1 to 100; Default: 20
q	query / string	Length 1 to 100; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.; Literal plain-text search. Omit for no search. SQL wildcard characters have no special meaning.
order	query / string	Allowed: asc, desc; Default: "desc"
sort	query / string	Allowed: createdAt, filename, size; Default: "createdAt"; Allowlisted field; arbitrary SQL column names are rejected.
contentType	query / string	Allowed: image/png, image/jpeg, image/webp

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/files?page=1&limit=20&order=desc&sort=createdAt&q=learn" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": [
    {
      "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "filename": "profile.png",
      "contentType": "image/png", "size": 1024, "createdAt": "2026-01-01T10:00:00.000Z",
      "downloadUrl": "/api/v1/files/b70efcda-37b8-47cc-915d-88e34902ae75/content"
    }
  ],
  "requestId": "req_example_01",
  "meta": {
    "page": 1, "limit": 20, "total": 1, "totalPages": 1, "hasNext": false,
    "hasPrevious": false
  }
}
```

RESPONSE TYPE data: File[]; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

POST

/api/v1/files

Upload a raster image as raw bytes

AUTH Bearer access token OR trusted-server API key; API key scope files:write
Requires files:write for API keys. Send the File/Blob directly, not JSON, base64, or multipart FormData. MIME, extension, signature, image structure and dimensions are checked. Upload size is limited by MAX_UPLOAD_BYTES; default 5 MiB. Maximum 40 million pixels and 16384 pixels per side. APNG and animated WebP are rejected. Files are private in Storage.

Parameter (* required)	Location / type	Allowed values and behavior
X-File-Name *	header / string	Length 1 to 120; Validated character format; Start with an ASCII letter/digit; remaining characters may be ASCII letters, digits, spaces, dots, underscores, and hyphens; no paths. Spaces are normalized to underscores. Extension must match MIME (case-insensitive).

REQUEST BODY Raw binary; Content-Type: image/png | image/jpeg | image/webp

Request example

```
curl -X POST "${API_BASE_URL}/api/v1/files" \
-H "Authorization: Bearer ${ACCESS_TOKEN}" \
-H 'X-File-Name: profile.png' \
-H 'Content-Type: image/png' \
--data-binary @profile.png
```

Response example - HTTP 201

```
{
  "success": true,
  "data": {
    "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "filename": "profile.png",
    "contentType": "image/png", "size": 1024, "createdAt": "2026-01-01T10:00:00.000Z",
    "downloadUrl": "/api/v1/files/b70efcda-37b8-47cc-915d-88e34902ae75/content"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: File; see the schema catalog. All examples are illustrative, not credentials.
RESPONSE HEADERS Location
ERROR STATUSES 400, 401, 403, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET

/api/v1/files/{id}

Read metadata for a file you own

AUTH Bearer access token OR trusted-server API key; API key scope files:read
Requires files:read for API keys.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/files/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "filename": "profile.png",
    "contentType": "image/png", "size": 1024, "createdAt": "2026-01-01T10:00:00.000Z",
    "downloadUrl": "/api/v1/files/b70efcda-37b8-47cc-915d-88e34902ae75/content"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: File; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

DELETE

/api/v1/files/{id}

Delete your file and its metadata

AUTH Bearer access token OR trusted-server API key; API key scope files:write
Requires files:write for API keys. PostgreSQL and Storage cannot share a transaction; a 503 FILE_DELETION_PENDING means access is already blocked and scheduled cleanup will resume. Retry is safe.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

REQUEST BODY None. Do not send a body.

Request example

```
curl -X DELETE "${API_BASE_URL}/api/v1/files/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {"id": "b70efcda-37b8-47cc-915d-88e34902ae75", "deleted": true},
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: Deleted; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET

/api/v1/files/{id}/content

Download your file as a binary attachment

AUTH Bearer access token OR trusted-server API key; API key scope files:read
Requires files:read for API keys. Successful responses contain bytes instead of a JSON envelope. Fetch with authorization, call response.blob(), and create an object URL. A normal image URL cannot attach your Bearer header.

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/files/${RESOURCE_ID}/content" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
HTTP 200
Content-Type: image/png | image/jpeg | image/webp
Content-Disposition: attachment; filename="profile.png"
X-Content-Type-Options: nosniff
Cache-Control: <VALUE>
```

Private image content. Response body uses the listed media type rather than a resource envelope.
RESPONSE HEADERS Content-Disposition, X-Content-Type-Options, Cache-Control
ERROR STATUSES 400, 401, 403, 404, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

USERS

GET /api/v1/users/me

Read your profile

AUTH Bearer access token only

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/users/me" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "id": "2cd65e90-ce9f-42e4-a7f4-1e6dc8dba84b", "email": "learner@example.com",
    "name": "API Learner", "role": "member", "createdAt": "2026-01-01T10:00:00.000Z",
    "updatedAt": "2026-01-01T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: User; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

PATCH /api/v1/users/me

Update your display name

AUTH Bearer access token only

Email, role, passwords, IDs and timestamps cannot be changed here.

JSON body (* required)	Type	Rules / defaults
name *	string	Length 1 to 100; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.

JSON object rejects unknown fields.

Request example

```
curl -X PATCH "${API_BASE_URL}/api/v1/users/me" \
-H "Authorization: Bearer ${ACCESS_TOKEN}" \
-H 'Content-Type: application/json' \
--data-binary @- <<'JSON'
{"name": "API Learner"}
JSON
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "id": "2cd65e90-ce9f-42e4-a7f4-1e6dc8dba84b", "email": "learner@example.com",
    "name": "API Learner", "role": "member", "createdAt": "2026-01-01T10:00:00.000Z",
    "updatedAt": "2026-01-01T10:00:00.000Z"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: User; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET /api/v1/users

List users (admin only)

AUTH Bearer access token only; admin role
Requires an active Bearer session and the admin role. API keys are never sufficient.

Parameter (* required)	Location / type	Allowed values and behavior
page	query / integer	Range 1 to 10000; Default: 1
limit	query / integer	Range 1 to 100; Default: 20
q	query / string	Length 1 to 100; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.; Literal plain-text search. Omit for no search. SQL wildcard characters have no special meaning.
order	query / string	Allowed: asc, desc; Default: "desc"
sort	query / string	Allowed: createdAt, name, email; Default: "createdAt"; Allowlisted field; arbitrary SQL column names are rejected.
role	query / string	Allowed: member, admin

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/users?page=1&limit=20&order=desc&sort=createdAt&q=learn" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": [
    {
      "id": "2cd65e90-ce9f-42e4-a7f4-1e6dc8dba84b", "email": "learner@example.com",
      "name": "API Learner", "role": "member", "createdAt": "2026-01-01T10:00:00.000Z",
      "updatedAt": "2026-01-01T10:00:00.000Z"
    }
  ],
  "requestId": "req_example_01",
  "meta": {
    "page": 1, "limit": 20, "total": 1, "totalPages": 1, "hasNext": false,
    "hasPrevious": false
  }
}
```

RESPONSE TYPE data: User[]; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

AUDIT

GET /api/v1/audit-logs

List audit events (admin only)

AUTH Bearer access token only; admin role
Requires an active Bearer session and the admin role. No token, password, secret, or raw IP is returned. Search q is not supported.

Parameter (* required)	Location / type	Allowed values and behavior
page	query / integer	Range 1 to 10000; Default: 1
limit	query / integer	Range 1 to 100; Default: 20
order	query / string	Allowed: asc, desc; Default: "desc"
sort	query / string	Allowed: createdAt; Default: "createdAt"; Allowlisted field; arbitrary SQL column names are rejected.
action	query / string	Length 1 to 100; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
actorId	query / string (uuid)	Optional; see request example.

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/audit-logs?page=1&limit=20&order=desc&sort=createdAt" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": [
    {
      "id": "b70efcda-37b8-47cc-915d-88e34902ae75",
      "actorId": "2cd65e90-ce9f-42e4-a7f4-1e6dc8dba84b", "action": "project.create",
      "resourceType": "projects", "resourceId": "b70efcda-37b8-47cc-915d-88e34902ae75",
      "requestId": "req_example_01",
      "metadata": {},
      "createdAt": "2026-01-01T10:00:00.000Z"
    }
  ],
  "requestId": "req_example_01",
  "meta": {
    "page": 1, "limit": 20, "total": 1, "totalPages": 1, "hasNext": false,
    "hasPrevious": false
  }
}
```

RESPONSE TYPE data: AuditEvent[]; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

SYSTEM

GET /health

Check API liveness

AUTH Public; no auth header

Request example

```
curl -X GET "${API_BASE_URL}/health"
```

Response example - HTTP 200

```
{"success": true, "data": {"status": "ok", "version": "1.0.0"}, "requestId": "req_example_01"}
```

RESPONSE TYPE data: object; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET /ready

Check database readiness

AUTH Public; no auth header

Runs a lightweight PostgreSQL query. A failed dependency returns 503 without infrastructure details.

Request example

```
curl -X GET "${API_BASE_URL}/ready"
```

Response example - HTTP 200

```
{"success": true, "data": {"status": "ready"}, "requestId": "req_example_01"}
```

RESPONSE TYPE data: object; see the schema catalog. All examples are illustrative, not credentials.

ERROR STATUSES 400, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET /openapi.json

Download this OpenAPI 3.1 document

AUTH Public; no auth header

Request example

```
curl -X GET "${API_BASE_URL}/openapi.json"
```

Response example - HTTP 200

```
HTTP 200
Content-Type: application/json
```

The OpenAPI document itself (no envelope). Response body uses the listed media type rather than a resource envelope.

ERROR STATUSES 400, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET /docs

Read the API reference

AUTH Public; no auth header

Request example

```
curl -X GET "${API_BASE_URL}/docs"
```

Response example - HTTP 200

```
HTTP 200
Content-Type: text/html | text/plain
```

Self-contained HTML locally; the free Supabase domain rewrites HTML to text/plain. Use the PDF or portable docs/index.html for the formatted reference. No JSON envelope. Response body uses the listed media type rather than a resource envelope.

ERROR STATUSES 400, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET /docs.css

Read the API reference stylesheet

AUTH Public; no auth header

Request example

```
curl -X GET "${API_BASE_URL}/docs.css"
```

Response example - HTTP 200

```
HTTP 200
Content-Type: text/css
```

Local CSS stylesheet (no JSON envelope). Response body uses the listed media type rather than a resource envelope.

ERROR STATUSES 400, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

GET

/api/v1

Discover API links and version

AUTH Public; no auth header

Request example

```
curl -X GET "${API_BASE_URL}/api/v1"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "version": "v1", "documentation": "/docs", "openapi": "/openapi.json",
    "resources": ["example"]
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: object; see the schema catalog. All examples are illustrative, not credentials.

ERROR STATUSES 400, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

API KEYS

GET

/api/v1/api-keys

List your API keys without their secrets

AUTH Bearer access token only

Request example

```
curl -X GET "${API_BASE_URL}/api/v1/api-keys" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "name": "Server integration",
        "prefix": "ak_AAAAAAAA",
        "scopes": ["resources:read"],
        "createdAt": "2026-01-01T10:00:00.000Z",
        "expiresAt": "2026-01-31T10:00:00.000Z"
      }
    ]
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: object; see the schema catalog. All examples are illustrative, not credentials.

ERROR STATUSES 400, 401, 403, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

POST

/api/v1/api-keys

Create a scoped key for a trusted server

AUTH Bearer access token only

The key secret is returned exactly once. Maximum 20 active keys per user. Copy it to your server secret store. Never put it in Vite/Next.js public variables, browser code, a mobile bundle, or a repository.

JSON body (* required)	Type	Rules / defaults
name *	string	Length 1 to 80; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.

JSON body (* required)	Type	Rules / defaults
scopes *	string[]	Items 1 to 4; Unique items; Values: resources:read, resources:write, files:read, files:write
expiresInDays	integer	Range 1 to 90; Default: 30

JSON object rejects unknown fields.

Request example

```
curl -X POST "${API_BASE_URL}/api/v1/api-keys" \
-H "Authorization: Bearer ${ACCESS_TOKEN}" \
-H 'Content-Type: application/json' \
--data-binary @- <<'JSON'
{"name": "Server integration", "scopes": ["resources:read"], "expiresInDays": 30}
JSON
```

Response example - HTTP 201

```
{
  "success": true,
  "data": {
    "id": "b70efcda-37b8-47cc-915d-88e34902ae75", "name": "Server integration",
    "prefix": "ak_AAAAAAAA",
    "scopes": ["resources:read"],
    "createdAt": "2026-01-01T10:00:00.000Z", "expiresAt": "2026-01-31T10:00:00.000Z",
    "key": "<API_KEY>"
  },
  "requestId": "req_example_01"
}
```

RESPONSE TYPE data: CreatedApiKey; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 409, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

DELETE

 /api/v1/api-keys/{id}

Revoke your API key

AUTH Bearer access token only

Parameter (* required)	Location / type	Allowed values and behavior
id *	path / string (uuid)	Optional; see request example.

REQUEST BODY None. Do not send a body.

Request example

```
curl -X DELETE "${API_BASE_URL}/api/v1/api-keys/${RESOURCE_ID}" \
-H "Authorization: Bearer ${ACCESS_TOKEN}"
```

Response example - HTTP 200

```
{"success": true, "data": {"revoked": true}, "requestId": "req_example_01"}
```

RESPONSE TYPE data: Revoked; see the schema catalog. All examples are illustrative, not credentials.
ERROR STATUSES 400, 401, 403, 404, 413, 415, 422, 429, 500, 503. Common error envelope and actions: Responses, lists and errors.

 Response schema catalog

Types below describe response data and pagination. All JSON examples in the endpoint reference retain their full success envelope. UUIDs are opaque server-generated identifiers; timestamps are UTC ISO 8601. Null means an explicitly absent value. Credential/hash/storage-owner internals never appear in normal resource DTOs.

User

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
email *	string (email)	Returned field
name *	string	Length 1 to 100; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
role *	string	Allowed: member, admin
createdAt *	string (date-time)	Returned field
updatedAt *	string (date-time)	Returned field

AuthTokens

Field (* always present)	Type	Meaning / constraints
user *	User	Fields: id, email, name, role, createdAt, updatedAt
accessToken *	string	Short-lived JWT. Keep in memory; send as Authorization: Bearer <accessToken>.
refreshToken *	string	Opaque, single-use secret; replaced on refresh.
tokenType *	string	Always "Bearer"
expiresIn *	integer	Range 1 to unbounded; Access lifetime in seconds; default 900.
sessionExpiresAt *	string (date-time)	Returned field

Session

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
createdAt *	string (date-time)	Returned field
expiresAt *	string (date-time)	Returned field
lastSeenAt *	string (date-time)	Returned field
current *	boolean	Returned field

ApiKey

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
name *	string	Length 1 to 80; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
prefix *	string	Returned field
scopes *	string[]	Values: resources:read, resources:write, files:read, files:write
createdAt *	string (date-time)	Returned field
expiresAt *	string (date-time)	Returned field

CreatedApiKey

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
name *	string	Length 1 to 80; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
prefix *	string	Returned field
scopes *	string[]	Values: resources:read, resources:write, files:read, files:write
createdAt *	string (date-time)	Returned field
expiresAt *	string (date-time)	Returned field
key *	string	Copy once. It cannot be retrieved again.

Project

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
name *	string	Length 1 to 120; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
description *	string	Length 0 to 2000; Default: ""; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
status *	string	Allowed: active, archived; Default: "active"
createdAt *	string (date-time)	Returned field
updatedAt *	string (date-time)	Returned field

Task

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
title *	string	Length 1 to 200; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
description *	string	Length 0 to 4000; Default: ""; Trimmed plain text. Angle brackets and unsafe control characters are rejected. Render as text in the frontend.
status *	string	Allowed: todo, in_progress, done; Default: "todo"
priority *	string	Allowed: low, medium, high; Default: "medium"
projectId *	string (uuid) null	Default: null; ID of a project you own, or null to remove the association.
dueDate *	string (date) null	Default: null; Valid calendar date, YYYY-MM-DD, or null.
createdAt *	string (date-time)	Returned field
updatedAt *	string (date-time)	Returned field

File

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
filename *	string	Length 0 to 120
contentType *	string	Allowed: image/png, image/jpeg, image/webp
size *	integer	Range 1 to unbounded
createdAt *	string (date-time)	Returned field
downloadUrl *	string	Relative protected URL: fetch it with your auth header.

AuditEvent

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
actorId *	string (uuid) null	Returned field
action *	string	Returned field
resourceType *	string	Returned field
resourceId *	string null	Returned field
requestId *	string	Returned field
metadata *	object	Returned field
createdAt *	string (date-time)	Returned field

PaginationMeta

Field (* always present)	Type	Meaning / constraints
page *	integer	Range 1 to 10000
limit *	integer	Range 1 to 100
total *	integer	Range 0 to unbounded
totalPages *	integer	Range 0 to unbounded
hasNext *	boolean	Returned field
hasPrevious *	boolean	Returned field

Revoked

Field (* always present)	Type	Meaning / constraints
revoked *	boolean	Always true

Deleted

Field (* always present)	Type	Meaning / constraints
id *	string (uuid)	Returned field
deleted *	boolean	Always true

Error

Field (* always present)	Type	Meaning / constraints
success *	boolean	Always false
error *	object	Fields: code, message, details
requestId *	string	Returned field

ErrorDetail

Field (* always present)	Type	Meaning / constraints
field *	string	Returned field
message *	string	Returned field

Local development and deployment

Frontend learners only need the live API URL and their own account. Backend contributors can run Node.js 22.12+ and npm locally with persisted PGLite (real PostgreSQL) and private filesystem storage. No cloud account, Docker or card is required for this local mode.

```
npm ci
npm run setup:local
npm run db:migrate
npm run db:seed
npm run dev
# Open http://localhost:8787/docs
```

setup:local creates ignored .env.local with four distinct random secrets and restrictive permissions, preserving existing values. Local seed creates learner@local.test/admin@local.test with a generated password printed once; rerunning rotates those local passwords and revokes sessions. Never seed production.

```
npm run check          # Types, PostgreSQL tests, OpenAPI and Edge bundles
npm run format:check
npm audit
npm run openapi:export # After editing src/openapi.ts
```

Deploy a separate installation on Supabase Free

Create a Free Supabase project. Set its projectRef/baseUrl in docs/configuration.json and OpenAPI server defaults, and configure exact frontend origins. Supabase supplies its database URL and private Storage credential inside the Edge runtime; never send these to a frontend.

```
npm run supabase:login
npx supabase link --project-ref YOUR_PROJECT_REF
npm run setup:production-secrets
npm run deploy
npm run maintenance:setup
```

setup:production-secrets creates or preserves four independent server secrets in an ignored permission-restricted backup and sends them to Supabase Secrets. deploy validates the project, checks secret names/configuration, applies tracked PostgreSQL migrations and deploys backend/maintenance with the API bundler. No Docker is required for deployment. The optional Docker-based Supabase local stack provides Edge runtime parity.

verify_jwt=false disables only the native Supabase JWT gate: protected REST routes still verify custom access JWTs, sessions and API-key scopes in the application. Native Supabase Auth tokens do not authenticate this REST API. The separate maintenance function checks its independent secret.

Private operator endpoint - outside the public REST base

POST <https://fvhpmujmbavwdpjyayal.supabase.co/functions/v1/maintenance>. Authorization: Bearer <MAINTENANCE_SECRET>; server/Cron only. No query parameters. Send no body or {} (maximum actual body 1024 bytes; Content-Type: application/json). Origin headers are prohibited; never call this from browser code.

```
HTTP 200
{"success":true,"data":{"completed":true},"requestId":"<REQUEST_ID>"}
```

401 invalid credential; 403 browser Origin; 405 non-POST with Allow: POST; 413 oversized body; 415 wrong type/encoding; 422 unexpected JSON; 503 configuration/dependency failure. Cleanup may make partial progress before 503; retry is safe. Separate contract: docs/maintenance-openapi.json.

Free plan and platform behavior

This deployment uses the Free plan. Included quotas are finite: 500 MB database, 1 GB file storage, 5 GB egress and 500,000 Edge Function invocations per month. Low-activity Free projects may pause after a week. Automatic backups and availability SLAs are not included. Check current Supabase pricing before expanding usage; no paid feature is needed for this learning setup.

Supabase rewrites hosted HTML responses to plain text on the standard free domain. Use this PDF, docs/index.html or the live OpenAPI JSON for integration reference; the styled local /docs page remains available. API calls use HTTPS normally and require no custom domain.

Supabase Cron invokes private maintenance every 15 minutes using pg_cron/pg_net and a token encrypted in Vault. Cleanup retries tombstones and scans orphans after a 24-hour grace period; batches are bounded and backlog-dependent. Monitor failures and quota usage. Application limits do not stop a rejected request from consuming a platform invocation.

GitHub main contains the source, exported contract and PDF. CI validates pushes; the manual deployment workflow uses encrypted SUPABASE_ACCESS_TOKEN, SUPABASE_PROJECT_REF and SUPABASE_DB_PASSWORD. Application secrets remain in Supabase. Operator admin promotion requires an existing active account and records an audit event; no default production admin exists.

```
npm run admin:promote -- you@example.com --production
```



Configuration and maintenance reference

Non-secret defaults come from docs/configuration.json. Supabase Secrets may override them. Only the complete API URL belongs in public frontend environment variables; never include server credentials in this PDF or browser code.

Setting	Production default	Purpose / range
ENVIRONMENT	production	development, test or production.
ALLOWED_ORIGINS	http://localhost:3000,http://localhost:5173,http://localhost:5500,http://127.0.0.1:3000,http://127.0.0.1:5173,http://127.0.0.1:5500	Exact comma-separated origins; configured loopback HTTP is allowed.
JWT_ISSUER	frontend-foundation-api	Expected access JWT issuer.
JWT_AUDIENCE	frontend-learning-app	Expected access JWT audience.
ACCESS_TOKEN_TTL_SECONDS	900	Access JWT lifetime, 60-900 seconds.
SESSION_TTL_SECONDS	604800	Absolute session lifetime, 3600-2592000 seconds; exceeds JWT TTL.

Setting	Production default	Purpose / range
MAX_JSON_BYTES	65536	Actual streamed JSON limit, 1024-1048576 bytes.
MAX_UPLOAD_BYTES	5242880	Actual streamed upload limit, 1024-10485760 bytes.
ALLOW_REGISTRATION	true	true or false; disable registration for closed applications.
AUDIT_RETENTION_DAYS	90	1-3650 days; scheduled deletion is bounded.

Server secrets and platform-injected variables

Name	Purpose / source
JWT_SECRET	Independent random HS256 signing secret; 43-1024 characters.
PASSWORD_PEPPER	Independent password pepper; 43-1024 characters.
IP_HASH_SECRET	Independent HMAC pseudonym secret; 43-1024 characters.
MAINTENANCE_SECRET	Independent random secret authorizing the private Cron function.
SUPABASE_URL	Platform-injected project API gateway URL.
SUPABASE_DB_URL	Platform-injected server database connection URL; private.
SUPABASE_SERVICE_ROLE_KEY	Platform-injected private Storage credential; bypasses RLS.
VITE_API_BASE_URL / NEXT_PUBLIC_API_BASE_URL	Frontend public variable: full live API URL including function path.
SUPABASE_ACCESS_TOKEN / SUPABASE_PROJECT_REF / SUPABASE_DB_PASSWORD	Operator/CI deployment environment only; never browser variables.

Generate secrets from at least 32 random bytes and keep them distinct. setup scripts preserve existing secrets. Changing the password pepper invalidates stored verifiers; plan a migration/reset. JWT-secret rotation invalidates access tokens. Supabase_* variables are injected by the platform and must not be manually submitted with that reserved prefix.

Capability	Behavior
PostgreSQL	Users, owned resources, sessions, hashed keys, transactional audits and atomic limits.
Private Storage	foundation-files bucket; no anonymous policy or public object URL.
Request limits	120 API requests/minute per ingress IP/user; 10 auth attempts/minute per IP/account. Fixed-window atomic counters.
Maintenance	Every 15 minutes; up to 100 tombstones/100 objects and 1000 rows per expired-row batch.

Security and extension boundaries

Passwords use peppered Argon2id v19 with a random 16-byte salt, 19 MiB memory, two iterations and one lane. The pinned embedded WASM heap is bounded below 32 MiB. Strict field allowlists, bound SQL, live database authorization, secure headers, explicit CORS and streamed byte limits guard requests. RLS and grants deny native anon/authenticated access to application tables. Frontend output encoding remains required.

Email verification, password recovery/reset, MFA, breached-password checks and CAPTCHA are future modules. An email identifier is not verified ownership. Extend route/controller -> service -> repository with strict schemas, owner-scoped SQL, migrations, auth/scope gates, transactional audit, OpenAPI and behavior tests. Define monitoring, backup and restore procedures for real production data.

Source documents and official references

OpenAPI JSON: <https://fvhpmujmbavwdpjyayal.supabase.co/functions/v1/backend/openapi.json>

Frontend client / guide: [examples/frontend.ts](#) / [docs/frontend.md](#)

Architecture / security: [docs/architecture.md](#) / [docs/security.md](#)

Deployment / configuration: [README.md](#) / [docs/configuration.json](#) / [supabase/config.toml](#)

Supabase pricing: <https://supabase.com/pricing>

Secrets / runtime: <https://supabase.com/docs/guides/functions/secrets>

Platform limits: <https://supabase.com/docs/guides/functions/limits>

Project pausing: <https://supabase.com/docs/guides/platform/free-project-pausing>

Password guidance: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html